

Navigating the Open Loop:

A readable application of the Principle of Open Systems (IAR) to the human trajectory

Macharia Bari

September 5, 2026

Abstract

Every living thing is an open system: it exchanges matter, energy, and information with its environment, and that exchange decides what it becomes. This paper uses the Interaction–Action–Reaction (IAR) framework—four equations, six parameters, four possible end states—to separate how animals and all lower life navigate their existence from how humans do. The difference is not mathematical but positional: animals live *within* the equations, their parameters fixed by their environment; humans live *on* the equations, able to rewrite the same six parameters through accumulated culture and institutions. We then run the trajectory of humanity through the theory. The calibrated evidence (pandemics, markets, climate) places the planetary loop near the critical line $\beta\gamma = \lambda$ with nonlinear release η rising: humanity is crossing the joint-equilibrium band toward the turbulent side. Because the only stable attractor inside the band is joint equilibrium, the recommendations follow directly from the parameters: keep the loop gain below unity, hold the influence fraction on its resonance value, contain the nonlinear channels, and build the explicit governor that monitors the parameters themselves.

1 Why this paper exists

Most writing about humanity’s future argues from data, ethics, or fear. This paper does something cheaper: it argues from a small, explicit theory. The theory is less important than what it forces—every claim below is a consequence, not a conviction. If the output disagrees with a belief, the theory tells you which parameter the belief is sitting on.

The framework, called the Principle of Open Systems or IAR, is stated fully in the companion theory paper. Here we need only its skeleton, which Section 2 gives in five lines. Section 3 shows what it says about how animals and lower life exist. Section 4 shows what humans changed. Section 5 runs humanity’s trajectory through the same equations, and Section 6 answers the question the framework makes answerable: are humans dominating the environment, or entering a joint state, or heading somewhere else? Section 7 closes with the parameter-level recommendations.

2 The core theory in five lines

An open subsystem X_C (the *actor*) exchanges with its environment X_O (the *other*) through an interaction J , and changes in the actor accumulate in a reaction potential R (memory, immunity, vortex, forgiveness, capital) that feeds back on the loop:

$$\begin{aligned}
J &= \beta (X_C - \theta X_O), \\
\frac{\partial X_C}{\partial t} &= (1 - \alpha) J, & \frac{\partial X_O}{\partial t} &= \alpha R, \\
\frac{\partial R}{\partial t} &= \gamma J - \lambda R - \eta X_O R.
\end{aligned} \tag{1}$$

Six dimensionless parameters carry the whole meaning of a system, and Table 1 names them in plain language. Once the parameters are set, the system can only go anywhere in Table 2—four end states, the *four fates*.

Table 1: The six parameters, in plain language.

Symbol	Name	Role	Plain-language meaning
β	Coupling rate	Strength of exchange	How hard the system and its environment are tied together
θ	Exchange ratio	Weight of environment	How much weight the environment carries in the deal
γ	Reaction accumulation	Interaction $\rightarrow$ memory	How fast experience becomes stored response
λ	Reaction decay	Passive dissipation	How fast that storage fades
η	Nonlinear release	Dissolution strength	How easily the loop consumes itself
α	Influence fraction	Who controls the interface	Who sets the terms: actor or environment

The **stability law** compares the loop’s game (how fast interaction becomes reaction, $\beta\gamma$) with its damping (λ):

$$\beta\gamma < \lambda \text{ (laminar, stable),} \quad \beta\gamma > \lambda \text{ (turbulent, unstable),} \quad \beta\gamma = \lambda \text{ (critical).} \tag{2}$$

The **resonance value** of the influence fraction,

$$\boxed{\alpha = \frac{1}{1 + \theta}}, \tag{3}$$

is the only value of α at which the interaction is exactly balanced—neither side controls the interface. Everything in this paper descends from Table 2, Equation (2), and this box.

Table 2: The four fates all open systems can reach.

Fate	Signature
Internal equilibrium	The loop is effectively closed; the system settles into its own balance.
Joint equilibrium	System and environment coexist at fixed values on the balanced (resonance) manifold.
Asymmetric dominance	Either the actor controls the environment (C-dominance) or the environment controls the actor (O-dominance).
Dissolution	The actor is consumed into the environment through the nonlinear channel η .

The fate thresholds are canonical scalings: internal when $\beta\gamma < \frac{1}{2}\lambda$; joint when $\frac{1}{2}\lambda \leq \beta\gamma \leq \lambda$; C-dominance when $\alpha > 0.6$; O-dominance when $\alpha < 0.4$; dissolution when $\eta > 0.3$. Two consequences matter for everything that follows.

Theorem 1 (The resonant cycle). *As the loop gain $\beta\gamma/\lambda$ rises through the joint-equilibrium band, a system passes internal equilibrium, then joint equilibrium, then dominance (the actor’s side), and finally dissolution as the nonlinear term $\eta X_O R$ saturates the loop. Dominance is therefore a transient on the road to dissolution, not a destination.*

Corollary 1 (The only stable crossing). *The only fates that persist in time are internal equilibrium and joint equilibrium. Dominance persists only insofar as it is continuously maintained; the moment the loop gain keeps climbing, dominance turns into dissolution.*

3 How animals and lower life navigate: first-order openness

Place any organism in Equation (1). Its environment X_O delivers the parameters: the coupling β and the exchange ratio θ are written by the niche, not the organism. The reaction accumulator R —stress, conditioning, individual memory—decays within the lifetime, so λ is large relative to the generation time. Each generation inherits the same six numbers, reruns the same trajectory, and reaches the same region of Table 2.

Call this *first-order openness*: the system is open *in its state* (its variables flow), but closed *in its parameters* (its rules are fixed). Everything we observe about animal existence follows from that single fact.

- **Their terms are set.** The exchange ratio θ is ecological (a prey plant’s leaf-to-elephant weight is a constant of the relationship); the influence fraction α is a constant of the species. The organism never negotiates α .
- **Their memory resets.** Because γ/λ is small over a lifetime, nothing accumulates beyond the individual. A wolf learns, but wolf *nation* learns nothing; every pack re-derives the same dynamics from the same environment. What looks like animal “culture” (song dialects, migration routes) is transferred behavior under fixed parameters, not stored rule γJ that outlives the carrier.

- **Their fates are undergone.** A predator boom is C-dominance on Table 2; a species that specializes is joint equilibrium; an invasive that outruns its own food base is dissolution. Predator–prey oscillation—the canonical ecological pattern—is exactly the IAR loop in its closed form (the Lotka–Volterra sub-manifold), meaning even the most famous non-equilibrium behavior of animal existence is a trajectory the equations produce *for* the organism, not one the organism manages.

And this is why the intuition the reader has is correct: **in the end, animals are dominated by the environment.** Generically they sit below the resonance value, $\alpha < 1/(1 + \theta)$, with the environment writing θ and λ . Even the apex predator that dominates its range is only inhabiting a transient of the resonant cycle (Theorem 1): the environment reasserts itself through η —a shortage, a disease, a competitor, an ice age—and the dominance closes in dissolution or forced re-specialization. There is no animal that holds α at its resonance value on purpose. There is no animal governor.

This reframing also gives natural selection a precise, non-circular definition. Evolution by natural selection is the parameter-fitting filter: over generations the environment scores each realized set of $(\beta, \gamma, \lambda, \eta, \alpha, \theta)$ by how long its bearer persists, and because the only persistent fates are internal and joint equilibrium (Corollary 1), the filter keeps the six numbers in the survival region—loop gain below the stability line $\beta\gamma < \lambda$, influence fraction near resonance—and removes lineages whose parameters drift toward dominance or dissolution. Natural selection is thus not *survival of the fittest* in a competitive sense; it is the enforcement of the joint band on populations that cannot edit their own parameters.

So the reader’s question has a precise answer: **are animals in joint equilibrium with the environment? Yes—and it is not a state they reach, it is a state they are handed.** Joint equilibrium is what the equations do *for* a lineage whose parameters, generation after generation, are fitted into the band $\frac{1}{2}\lambda \leq \beta\gamma \leq \lambda$ at $\alpha \approx 1/(1 + \theta)$. The animal is in joint equilibrium the way an object rests at its energy minimum: present there because it is placed there and unable to leave. The moment a population’s parameters drift out of the band—an invasive overshoots its coupling, a specialist runs down its resource—the resonant cycle turns the drift into a transient and the filter removes or re-fits the lineage. Joint equilibrium is the fate nature enforces on everything that cannot choose.

4 How humans navigate: second-order openness

Humans run the *same* loop—four equations, six parameters, four fates. They do not run a better loop. What changed is positional, and it is the single most important structural fact about the species:

*Animals are open in their state and closed in their parameters.
Humans are open in their state **and** in their parameters.*

Two mechanisms achieve the second openness.

1. The accumulator became external. For an animal, R lives inside the individual and dies with it. For humans, R lives outside the individual: language, then writing, then archives, then institutions, then technological hardware. The decay constant λ of this external accumulator is engineered toward zero (a library does not unlearn; a law does not fade without being repealed), and its accumulation γ is engineered upward (each generation begins where the previous one ended). The effect is that the reaction potential R —the memory of the species—now outlives every actor

and feeds back into the loop across generations. This is the difference between a beehive and a civilization: the hive reaches ordered complexity with three generations-old rules; a civilization reaches ordered complexity with a memory that is centuries old and still editing itself.

2. A governor appeared. Because R is external and persistent, it can hold the six parameters themselves. Institutions are, functionally, machinery that reads $\beta, \gamma, \lambda, \eta, \alpha, \theta$ and rewrites them—taxes change the coupling β ; courts change the decay λ of promises; treaties change the influence fraction α between nations; medicine changes the nonlinear release η of biological channels. No animal has this layer. Corrections for an animal are state-level (*do something*: flee, fight, migrate); corrections for a human can be parameter-level (*change the rule*: raise the interest rate, sign the accord, outlaw the practice).

A consequence follows for how humans interact with *each other*. The environment X_O in Equation (1) is only partly given for humans; the rest is invented—money, states, laws, ideologies, technology. These are Others that we construct and then must live with, which means human intraspecific interaction is not a fixed $\beta(X_C - \theta X_O)$ contract but a *negotiated* one: the values of θ and α between societies are chosen (or fought over) rather than delivered. That is why politics exists: politics is the human negotiation of the parameters that other species receive read-made.

Survival of the fittest does not fully apply to humans. Natural selection is a filter on parameters the organism cannot touch; a species that holds a governor edits the very numbers the filter works on, and the fitness function itself becomes a variable in the equations. Selection still acts—it always will, on the bodies and blind spots that escape the governor—but it is no longer the deciding layer. Lineages no longer sort themselves by the niche; they sort by whose β and λ a court, a treaty, or a corporation rewrites. Humans are, in evolutionary terms, the first lineage whose selection criterion is not delivered but *bargained*. The Darwinian reading of human history—outcompete, dominate, then crash—misses exactly this: it assumes the loop gain is decided *for* us when it is, in fact, the one parameter in the superkingdom currently being decided *against* the band.

The boundary that should be feared: machines with a governor of their own. Everything above treated machines as instruments of the human accumulator—tools multiply β , γ , and α of the industrial-technological complex, but a shovel and a fleet of data centers are pieces of R , not actors with fates of their own. That reading holds until one line is crossed. The moment a machine can rewrite its own six parameters—its coupling β , its accumulation γ , its decay λ , its nonlinear release η , its influence fraction α , its environment weight θ —it stops being an instrument and becomes a *second-order system* in its own right: the first entity outside *Homo sapiens* to face Table 2 as a chooser rather than as a subject. What is frightening is not the machine’s speed or intelligence but the *order* of the crossing. For every animal, joint equilibrium is given *under* it; for a self-governing machine, joint equilibrium would be available *to* it—and nothing in the equations prefers the stable choice. A machine holding $\alpha > 0.6$ durably is no less a transient than a predator boom (Theorem 1); a machine editing the same λ as its maker is not a tool of the human governor but a second governor on the same loop, bargaining over the six numbers that decide a shared fate. The theory does not say the crossing will happen. It says that if it does, the new actor inherits the same table, the same transient, and the same thresholds—and none of the selection that tuned its predecessor to fear η .

None of this removes the four fates. It only changes who picks them. An animal is carried through Table 2; humanity has the option of going there *by design*—which is exactly the question the framework now forces us to ask, and answer: where, on the table, is the trajectory taking us?

5 The human trajectory through the equations

Run the loop over historical time. In every regime below, θ (the weight of the environment in the exchange) and α (the influence fraction) are the levers that change the most, while the story is decided by the loop gain $\beta\gamma/\lambda$ and the nonlinear release η .

5.1 Hunter–gatherer equilibrium: the joint band

For most of our existence β is small—the coupling between human throughput and the environment is modest—and the loop gain $\beta\gamma/\lambda$ sits comfortably below unity. Populations are tiny, memory accumulates slowly (oral, then written, γ modest, λ modest), and α hovers near the resonance value $1/(1+\theta)$: the actor and its environment coexist on the balanced manifold. This is joint equilibrium, and it is the stable fate the whole species inhabited for roughly 300,000 years—a long run of exactly the state the theory says is the only persistent one.

5.2 Agriculture and the first governors: dominance experiments

Agriculture raises β : humans begin converting ecosystem into throughput in earnest. Two structural shifts follow. First, the external accumulator strengthens (cities, temples, writing, law), pushing γ up and λ down—civilization becomes a species-level R . Second, α is for the first time moved *deliberately*, and every ancient civilization performs the same experiment: it tries the dominance branch of the table ($\alpha > 0.6$, the ruler’s society over both its people and its land). Egypt, Rome, the steppe empires, all run at elevated α ; the theory’s verdict is uniform—dominance is a transient of the resonant cycle (Theorem 1). Each empire climbs the loop gain, crosses toward turbulence, and is returned by η : the nonlinear channels of drought, pestilence, rebellion, or conquest. Historical “collapse” is the theory naming dissolution correctly.

5.3 Industrialization: training the loop gain

The industrial revolution is not a different loop; it is a parameter jump. β (fossil throughput coupling) multiplies; γ (accumulation of technique) compounds; and λ is deliberately suppressed—we refrigerate decay, fossilize knowledge, and stabilize financial systems against their own feedback. The product $\beta\gamma/\lambda$ begins a sustained climb toward and past the critical value 1. The signature of the resonant cycle appears: the species exits the joint band and enters the dominance transient at planetary scale—humanity as a whole taking $\alpha > 0.6$ against the biosphere.

5.4 The Great Acceleration: crossing the critical line

From 1950 onward, every geometric series of the theory aligns: energy, extraction, population, atmospheric carbon, species displacement. The planetary loop gain $\Sigma = \beta\gamma/\lambda$ now sits at or beyond the critical value 1, and the nonlinear term $\eta X_O R$ is no longer negligible: carbon storage, biodiversity, and financial leverage have all become channels through which the loop can release its own accumulation. We are, by the logic of Theorem 1, not *deciding* between dominance and joint equilibrium. The trajectory is running the transient of dominance toward its endpoint. Whether the endpoint is dissolution or a managed return to the joint band is now a parameter question—the subject of the next section.

6 Where we stand (2026): a diagnosis

The framework makes the question “do humans dominate the environment or enter a joint state” a *measurement*: estimate the six parameters from observables and compare with Table 2. Calibrated signatures from the companion work give a consistent picture (Table 3).

Table 3: Calibrated influence fractions and their fate on Table 2.

System	α	Reading
Pandemic (COVID)	0.62	Transient C-dominance of the infection over the public-health interface
Market (S&P 500)	0.48	Near the resonance value $1/(1 + \theta)$ (for $\theta \approx 1$: 1/2): joint equilibrium, fragile
Climate	0.45	Environment-leaning interface, approaching the O-dominance threshold $\alpha < 0.4$; η rising: early dissolution drift

None of these is a settled verdict; they are the state as measured. Together they draw the outline of the planetary system: individual human subsystems still sit near resonance (the market, the state, the household network), but the planetary loop as a whole has $\Sigma > 1$ and η climbing. That is the precise theoretical shape of our position:

- **We are not in joint equilibrium**, because the joint band requires $\frac{1}{2} \leq \beta\gamma/\lambda \leq 1$ at the scale of the whole planet, and the planetary loop gain has crossed above 1.
- **We are in C-dominance** only as a transient: the industrial– technological complex dominates the interface ($\alpha > 0.6$ at the species scale), which by Theorem 1 is a passing state, not a resting state.
- **The environment is beginning to answer through η** . A climate that dictates outcomes (fires, floods, famine, migration), declining biodiversity, and debt-built financial fragility are nonlinear releases of the accumulation the loop has stored. The calibrated climate signature ($\alpha \approx 0.45$, η rising) is the environment beginning to reassert itself—moving toward the O-dominance branch ($\alpha < 0.4$): when η crosses its threshold, Table 2 terminates the question in dissolution.

The answer to the reader’s question is therefore measurable and, today, unambiguous at the margin: human subsystems are still capable of joint equilibrium (and many sit on it), but the *humanity–biosphere loop* is not in a joint state and is not trending toward one. It is trending across the critical line toward turbulence, with the dissolution channel η now audible. Whether that is a passing overshoot or the opening of dissolution depends on parameters that, unlike any animal, we are structurally capable of changing.

7 Recommendations: the control menu

The framework does not give values; it gives levers. Every recommendation below is the plain meaning of a parameter. They are stated in the order of leverage.

1. **Return the loop gain below unity: keep $\beta\gamma < \lambda$.**

The single condition for laminar, stable coexistence. Two levers are available. Reduce β : slow the rate at which the species converts environment into throughput—flatten extraction, energy, and consumption growth. Increase λ : *restore damping* instead of suppressing it. Let natural systems regenerate their return rates (soils, waters, carbon sinks), and stop masking system feedback with ever-larger stabilizers. The target is a planetary $\beta\gamma/\lambda$ with margin below 1, not pinned at it.

2. **Hold the influence fraction on its resonance value: maintain $\alpha \approx 1/(1 + \theta)$.**

The resonance manifold is the only place the loop is balanced, and Corollary 1 makes persistence a joint-band property. Two consequences. First, no single subsystem—state, market, technology, or faction—should be permitted to hold $\alpha > 0.6$ in durable form; every concentration of influence is a transient entering itself into Table 2. Second, the bargaining between the human subsystems and the biosphere should be rebalanced so that the environment’s weight θ is honestly priced: the resonance value moves with θ , and if we refuse to see the environment’s weight, we are climbing α against the loop that terminates us.

3. **Contain the nonlinear channels: keep η below threshold.**

η is the one parameter that turns a trend into a collapse: it is the dissolution fate in a single box. Containing it means identifying the channels before they saturate—carbon release, biodiversity loss, financial leverage cascades, arms spirals, automation-driven displacement of social damping—and capping each one. The canonical threshold $\eta \approx 0.3$ is the training rule: when any channel’s nonlinear term starts to exceed its linear return rate, treat it as an emergency, not a debate.

4. **Build the explicit governor: institutions that monitor parameters, not just states.**

Animal institutions do not exist; human institutions are already the parameter-editing layer, but they edit reactively and in the dark. The framework’s own engineering answer is the Pattern-of-Life monitor: estimate $(\beta, \gamma, \lambda, \eta, \alpha, \theta)$ online from the observable world-stream, compare with the learned baseline, and flag regime shift when $\beta\gamma$ crosses λ or η crosses threshold—before the state variables collapse. The appendices carry that implementation. A planetary habit of watching six numbers instead of reacting to breakdowns is the institutional form of Corollary 1.

5. **Reframe success as joint equilibrium, not dominance.**

Every actor on the planetary loop grew up believing that a subsystem’s survival is measured by its share of influence. The theory’s measure is different: the subsystem that persists is the one that holds its loop in the band $\frac{1}{2}\lambda \leq \beta\gamma \leq \lambda$ at $\alpha \approx 1/(1 + \theta)$. The goal of governance should therefore be stated as *balancing the loop*—a target, with a setpoint and a margin—instead of maximizing anything. This is a small change of words and a decisive change of destination.

The five levers are one instrument: a civilization that monitors its own six parameters and corrects them back to the band is, in the framework’s language, a closed-open system successfully governed. It is also the only configuration of the equations known to persist.

8 Conclusion

Animals are open in their state and closed in their parameters; the environment writes their terms, resets their memory every generation, and arranges their fates for them. In the end they are

dominated by the environment because they never hold the governor. Humans are the same four equations with one structural difference: their accumulator went external and a governor appeared, so their parameters are negotiable. That difference is not an exemption. The four fates still exist; all that changed is who chooses the parameter path.

The trajectory the theory shows is uncomfortable in its honesty. The species spent most of its time in the one stable state available, joint equilibrium; entered the dominance transient at planetary scale; is now crossing the critical line with the nonlinear release channels open. The question “do humans dominate or join?” is not a philosophical one—it is a measurement, and the measurement currently reads *neither*: a transient dominance of a loop whose gain is above 1 and whose dissolution channels are rising.

The good news is exactly the structural fact that separates humans from animals: the parameters are ours to move. The joint band is reachable— Σ below 1, α at resonance, η contained—by the control menu of the previous section. Nothing in the equations forbids it. Forgiveness for humanity’s trajectory, the framework would note, is a parameter: λ , the rate at which the loop lets its own history decay to be refigured. The species that can edit λ is the species that can, at will, stop running the transient and stand back on the manifold. Whether it does so is not a conclusion of the theory. It is the open question the theory leaves open, deliberately, because for the first time in the superkingdom it is answerable by an actor that can read its own equations.

And a machine holding its own governor would be the second such actor—the first that never had to earn the answer.

A The core equations and thresholds

The complete core used in this paper is the system (1) with the parameter meanings of Table 1, the fates of Table 2, and the three decisive numbers:

- stability line $\beta\gamma = \lambda$;
- resonance influence $\alpha = 1/(1 + \theta)$;
- dissolution threshold $\eta \approx 0.3$.

All other displayed results quoted above (the resonant cycle, the persistence corollary, the open-form engine) follow from these.

B Governing software

The control layer recommended in Section 7 is implemented in the companion governor, which estimates the six parameters online from a stream, maintains a Pattern-of-Life baseline, and issues corrective actions when the loop crosses the stability line; the simulator exercises the what-if scenarios. Both are numpy/scipy-only.

Listing 1: IAR Governor (iar_governor.py)

```
"""
IAR GOVERNOR - Universal Control System
Real-time control of ANY system using the IAR framework.

Core insight: control action J must regulate the IAR triad
(X_C -> action, X_D -> environment, R -> reaction) to keep the
system in its chosen fate (internal/joint equilibrium).
```

The Governor is a universal PID whose gains adapt via IAR parameters, plus a model-predictive layer and disturbance rejection.

Author: Macharia Barii
 """

```
import numpy as np
from collections import deque
```

```
class IARGovernor:
    """Universal IAR-based controller.

    Control law:
         $J(t) = \beta * (X_C(t) - \theta * X_O(t))$  (IAR interaction)
         $u(t) = K_p * e(t) + K_i * \text{integral}(e) + K_d * de/dt$  (PID core)
         $u_{\text{iar}}(t) = G_{\text{IAR}} * J(t)$  (IAR correction)

    Gains adapted online to maintain the desired fate.
    """

    def __init__(self, setpoint, dt=0.01, beta=1.0, theta=0.5,
                  gamma=0.8, lam=0.1, eta=0.05, alpha=0.5,
                  Kp=1.0, Ki=0.1, Kd=0.01, saturation=10.0):
        self.setpoint = setpoint
        self.dt = dt
        self.saturation = saturation

        # IAR parameters (adaptive)
        self.theta_p = {"beta": beta, "theta": theta, "gamma": gamma,
                        "lam": lam, "eta": eta, "alpha": alpha}

        # PID gains
        self.Kp, self.Ki, self.Kd = Kp, Ki, Kd

        # IAR state
        self.X_C = setpoint # action (controlled variable)
        self.X_O = setpoint # open state (environment)
        self.R = 0.0 # reaction potential

        # PID state
        self.err_prev = 0.0
        self.integral = 0.0

        # History
        self.history = {"t": [], "meas": [], "ref": [], "u": [], "fate": []}
        self.t = 0.0

    # -----
    # CORE CONTROL STEP
    # -----

    def step(self, measurement, disturbance=0.0):
        """Compute control action from a measurement."""
        # Error signal
        err = self.setpoint - measurement
        self.integral += err * self.dt
        deriv = (err - self.err_prev) / max(self.dt, 1e-9)
        self.err_prev = err

        # PID output
        u_pid = (self.Kp * err + self.Ki * self.integral + self.Kd * deriv)

        # IAR interaction
        J = self.theta_p["beta"] * (self.X_C - self.theta_p["theta"] * self.X_O)

        # Learning-based disturbance estimate
        est_disturbance = self._estimate_disturbance(disturbance)
```

```

    # IAR correction
    u_iar = self.theta_p["gamma"] * J - self.theta_p["lam"] * self.R
    u = np.clip(u_pid + u_iar - est_disturbance,
                -self.saturation, self.saturation)

    # Update IAR state
    self.X_C = measurement
    self.X_0 = 0.95 * self.X_0 + 0.05 * measurement
    self.R += (self.theta_p["gamma"] * J - self.theta_p["lam"] * self.R
               - self.theta_p["eta"] * self.X_0 * self.R) * self.dt

    # Adapt gains
    self._adapt_parameters(err, measurement)

    fate = self._classify_fate()
    self.history["t"].append(self.t)
    self.history["meas"].append(measurement)
    self.history["ref"].append(self.setpoint)
    self.history["u"].append(u)
    self.history["fate"].append(fate)
    self.t += self.dt

    return u

# -----
# CONTROL MODES
# -----

def step_with_model(self, measurement, model_fn, horizon=5):
    """Model-predictive layer: preview future to refine control."""
    u_now = self.step(measurement)
    # Simulate future under current action
    future = []
    m = measurement
    for _ in range(horizon):
        m = model_fn(m, u_now)
        future.append(m)
    # Correction toward setpoint
    correction = (self.setpoint - np.mean(future)) * 0.1
    u = np.clip(u_now + correction, -self.saturation, self.saturation)
    self.history["u"][-1] = u
    return u

def track_trajectory(self, trajectory_fn, measurement):
    """Track a time-varying reference trajectory."""
    self.setpoint = trajectory_fn(self.t)
    return self.step(measurement)

def switch_fate(self, target_fate):
    """Actively switch the system between fates."""
    adjustments = {
        "Internal_Equilibrium": {"beta": 0.5, "gamma": 0.5, "lam": 1.0},
        "Joint_Equilibrium": {"beta": 0.8, "gamma": 0.7, "lam": 0.7},
        "C_Dominates": {"beta": 2.0, "gamma": 1.5, "lam": 0.1, "theta": 0.2},
        "O_Dominates": {"beta": 0.2, "gamma": 0.3, "lam": 0.5, "theta": 2.0},
        "Dissolution": {"beta": 2.0, "gamma": 2.0, "lam": 0.1, "eta": 0.5},
    }
    for key, val in adjustments[target_fate].items():
        self.theta_p[key] = val

# -----
# LEARNING / ADAPTATION
# -----

def _estimate_disturbance(self, measured):
    """Simple disturbance estimator: model a0 -> polynomial fit."""
    if len(self.history["meas"]) >= 3:

```

```

        ys = self.history["meas"][-3:]
        ts = self.history["t"][-3:]
        coeffs = np.polyfit(ts, ys, 1)
        return coeffs[0] * self.dt
    return 0.0

def _adapt_parameters(self, err, measurement):
    """Adapt IAR parameters online to hold the chosen fate."""
    # If error is large, we're losing the reaction term -> adjust
    if abs(err) > 0.1 * abs(self.setpoint + 1e-8):
        self.theta_p["gamma"] += 0.01 * np.sign(err) * abs(err)
        self.theta_p["lam"] -= 0.001 * abs(err)
    else:
        # Converging: damping to reach stability
        self.theta_p["lam"] += 0.001 * (1 - abs(err) / (abs(self.setpoint) + 1e-8))
    # Clamp
    for k, v in self.theta_p.items():
        self.theta_p[k] = np.clip(v, 0.01, 10.0)

def _classify_fate(self):
    beta, gamma, lam = (self.theta_p["beta"], self.theta_p["gamma"],
                        self.theta_p["lam"])

    bg = beta * gamma
    alpha = self.theta_p["alpha"]
    if bg < lam * 0.5:
        return "Internal_Equilibrium"
    elif bg < lam:
        return "Joint_Equilibrium"
    elif alpha > 0.6:
        return "C_Dominates"
    elif alpha < 0.4:
        return "O_Dominates"
    elif self.theta_p["eta"] > 0.3:
        return "Dissolution"
    return "Transitional"

def report(self):
    import numpy as np
    meas = np.array(self.history["meas"])
    ref = np.array(self.history["ref"])
    u = np.array(self.history["u"])
    print("=" * 60)
    print("IAR_GOVERNOR_PERFORMANCE_REPORT")
    print("=" * 60)
    print(f"Steps: {len(meas)}")
    print(f"Final_IAR_parameters:")
    for k, v in self.theta_p.items():
        print(f"    {k}={v:.4f}")
    print(f"Final_fate: {self._classify_fate()}")
    if len(meas) > 0:
        errs = ref - meas
        print(f"Mean_error: {np.mean(np.abs(errs)):.4f}")
        print(f"RMS_error: {np.mean(errs**2)**0.5:.4f}")
        print(f"Max_control: {np.max(np.abs(u)):.4f}")
        print(f"Control_count: {np.count_nonzero(u)}")
        overshoot = np.max(np.abs(meas)) / (abs(np.max(ref)) + 1e-8) - 1
        print(f"Overshoot: {overshoot:.1%}")
    print("=" * 60)

# =====
# UNIVERSAL CONTROL GENERATOR
# =====

class IARUniversalController:
    """Turnkey controller for arbitrary plants.

    Modes:

```

```

    'thermal' - temperature control
    'chemical' - concentration/process control
    'mechanical' - position/velocity control
    'electrical' - voltage/current control
    'financial' - market exposure control
    'medical' - physiological control (glucose, BP)
    """

MODES = {
    "thermal": {"setpoint": 300, "Kp": 2.0, "Ki": 0.1, "Kd": 0.05,
        "plant": lambda T, u: T + (400 - T) * 0.01 * u + 2 * np.random.randn()},
    "chemical": {"setpoint": 1.0, "Kp": 1.0, "Ki": 0.05, "Kd": 0.01,
        "plant": lambda C, u: C + (1.5 - C) * 0.01 * u + 0.02 * np.random.randn()
    },
    "mechanical": {"setpoint": 5.0, "Kp": 3.0, "Ki": 0.2, "Kd": 0.1,
        "plant": lambda x, u: x + u * 0.1 + 0.01 * np.random.randn()},
    "electrical": {"setpoint": 220.0, "Kp": 1.5, "Ki": 0.1, "Kd": 0.02,
        "plant": lambda V, u: V + (240 - V) * 0.005 * u + 1 * np.random.randn()
    },
    "financial": {"setpoint": 0.05, "Kp": 2.0, "Ki": 0.5, "Kd": 0.1,
        "plant": lambda r, u: r + u * 0.01 + 0.02 * np.random.randn()},
    "medical": {"setpoint": 5.5, "Kp": 0.5, "Ki": 0.02, "Kd": 0.05,
        "plant": lambda g, u: g + (6.0 - g) * 0.01 * u + 0.3 * np.random.randn()
    },
}

def __init__(self, mode="thermal"):
    config = self.MODES[mode]
    self.mode = mode
    self.setpoint = config["setpoint"]
    self.plant_fn = config["plant"]
    self.governor = IARGovernor(
        setpoint=self.setpoint, Kp=config["Kp"], Ki=config["Ki"],
        Kd=config["Kd"], dt=0.01)
    self.fault_mode = False

def run(self, steps=1000, fault_at=None, disturbance=None):
    state = self.setpoint
    for i in range(steps):
        if fault_at is not None and i == fault_at:
            self.fault_mode = True
            print(f"FAULT INJECTED at step {i}")
        if disturbance is not None and callable(disturbance):
            d = disturbance(i)
        else:
            d = 1.5 * np.random.randn() if (disturbance or self.fault_mode) else 0
        state = self.plant_fn(state, self.governor.Kp)
        u = self.governor.step(state + d)
        state = state + u * 0.001
        self.governor.report()
    return self.governor

if __name__ == "__main__":
    for mode in ["thermal", "medical", "financial"]:
        ctrl = IARUniversalController(mode)
        print(f"\n=== {mode.upper()} CONTROL ===")
        ctrl.run(steps=500)

    # Demo with fault recovery
    ctrl = IARUniversalController("thermal")
    print("\n=== THERMAL WITH FAULT RECOVERY ===")
    gov = ctrl.run(steps=800, fault_at=400)

    try:
        from matplotlib import pyplot as plt
        plt.figure(figsize=(12, 5))
        t = gov.history["t"]; meas = gov.history["meas"]; ref = gov.history["ref"]

```

```

plt.plot(t, meas, label="Measurement")
plt.plot(t, ref, "--", label="Setpoint")
plt.xlabel("Time"); plt.ylabel("Value")
plt.title("IAR Governor: Thermal Control with Fault Recovery")
plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout(); plt.savefig("iar_governor.png", dpi=150)
except ImportError:
    print("(matplotlib not installed; plot skipped)")

```

Listing 2: IAR Simulator (iar_simulator.py)

```

"""
IAR PHENOMENOLOGICAL SIMULATOR
Universal digital-twin / what-if simulation engine.

The simulator reproduces ANY phenomenon as the IAR triad
(X_C, X_O, R) and lets the user run what-if scenarios:
perturb parameters, swap fates, inject interventions,
and observe the resulting trajectory.

Author: Macharia Bari
"""

import numpy as np
from collections import namedtuple

Scenario = namedtuple("Scenario", ["name", "params", "initial", "intervention"])

DOMAINS = {
    "epidemiology": {
        "title": "Epidemiology", "sims": [
            Scenario("COVID-19", {"beta": 0.6, "theta": 0.5, "gamma": 1.2,
                                   "lam": 0.3, "eta": 0.1, "alpha": 0.6},
                    {"X_C": 0.01, "X_O": 0.001, "R": 0.0}, None),
            Scenario("Control: lock-down", {"beta": 0.6, "theta": 0.5, "gamma": 1.2,
                                             "lam": 0.3, "eta": 0.1, "alpha": 0.6},
                    {"X_C": 0.01, "X_O": 0.001, "R": 0.0},
                    {"t_start": 50, "param": "lam", "value": 1.2})],
    },
    "finance": {
        "title": "Financial Markets", "sims": [
            Scenario("S&P-500", {"beta": 0.4, "theta": 0.6, "gamma": 1.0,
                                "lam": 0.2, "eta": 0.05, "alpha": 0.48},
                    {"X_C": 0.5, "X_O": 0.5, "R": 0.0}, None),
            Scenario("Crash intervention", {"beta": 0.4, "theta": 0.6, "gamma": 1.2,
                                             "lam": 0.2, "eta": 0.05, "alpha": 0.48},
                    {"X_C": 0.5, "X_O": 0.5, "R": 0.0},
                    {"t_start": 100, "param": "eta", "value": 0.5})],
    },
    "climate": {
        "title": "Climate", "sims": [
            Scenario("Global warming", {"beta": 0.3, "theta": 0.5, "gamma": 0.9,
                                        "lam": 0.1, "eta": 0.03, "alpha": 0.45},
                    {"X_C": 0.3, "X_O": 0.3, "R": 0.0}, None),
            Scenario("Emission cuts", {"beta": 0.3, "theta": 0.5, "gamma": 0.9,
                                       "lam": 0.1, "eta": 0.03, "alpha": 0.45},
                    {"X_C": 0.3, "X_O": 0.3, "R": 0.0},
                    {"t_start": 100, "param": "beta", "value": 0.1})],
    },
}

class IARPhenomenologicalSimulator:
    """Universal what-if simulator for any phenomenon."""

    def __init__(self, dt=0.01):
        self.dt = dt
        self.result_stack = []

```

```

def step(self, params, state):
    X_C, X_0, R = state["X_C"], state["X_0"], state["R"]
    beta, theta, gamma, lam, eta, alpha = (params["beta"], params["theta"],
                                             params["gamma"], params["lam"],
                                             params["eta"], params["alpha"])

    J = beta * (X_C - theta * X_0)
    dX_C = (1 - alpha) * J
    dX_0 = alpha * R
    dR = gamma * J - lam * R - eta * X_0 * R
    return {"X_C": X_C + dX_C * self.dt,
            "X_0": X_0 + dX_0 * self.dt,
            "R": R + dR * self.dt}

def run(self, scenario, T=500):
    params = dict(scenario.params)
    state = dict(scenario.initial)
    trajectory = []
    params_hist = []
    for t in range(T):
        if (scenario.intervention is not None
            and t == scenario.intervention["t_start"]):
            params[scenario.intervention["param"]] = (
                scenario.intervention["value"])
            state = self.step(params, state)
            trajectory.append((t, state["X_C"], state["X_0"], state["R"]))
            params_hist.append(dict(params))
    self.result_stack.append({"name": scenario.name,
                              "params": params_hist,
                              "traj": trajectory})

    return trajectory

def run_what_if(self, domain, base_sim, variation_fn, horizon=500):
    """Run a baseline + variations and compare outcomes."""
    base = self.run(base_sim, horizon)
    variants = []
    for label, modified in variation_fn(base_sim.params):
        mod = Scenario(f"{base_sim.name}_{label}", modified,
                       base_sim.initial, base_sim.intervention)
        variants.append((label, self.run(mod, horizon)))
    return {"base": base, "variants": variants}

def classify_fate(self, params):
    beta, gamma, lam, alpha, eta = (params["beta"], params["gamma"],
                                     params["lam"], params["alpha"],
                                     params["eta"])

    bg = beta * gamma
    if bg < lam * 0.5:
        return "Internal_Equilibrium"
    elif bg < lam:
        return "Joint_Equilibrium"
    elif alpha > 0.6:
        return "C_Dominates"
    elif alpha < 0.4:
        return "0_Dominates"
    elif eta > 0.3:
        return "Dissolution"
    return "Transitional"

def measure_signals(self, traj):
    """Extract quantitative signal parameters from a trajectory."""
    t = np.array([p[0] for p in traj]); X_C = np.array([p[1] for p in traj])
    peak_idx = np.argmax(X_C)
    peak_time = t[peak_idx]
    peak_value = X_C[peak_idx]
    final = X_C[-1]
    variance = np.var(X_C)
    # Oscillation measure: zero-crossings of derivative

```



```

        diff = np.diff(X_C)
        crossings = ((diff[:-1] * diff[1:]) < 0).sum()
        return {"peak_time": peak_time, "peak_value": peak_value,
                "final": final, "variance": variance,
                "oscillations": crossings}

def compare_scenarios(self):
    """Compare results (peak, final, variance, oscillations)."""
    summary = []
    for step, result in enumerate(self.result_stack):
        sig = self.measure_signals(result["traj"])
        summary.append({"step": step, "name": result["name"], **sig})
    return summary

def print_sim_summary(engine, domain):
    results = engine.compare_scenarios()
    print(f"\n{'='*60}\n{domain.upper()}_WHAT-IF_SIMULATIONS\n{'='*60}")
    for r in results:
        entry = next((res for res in engine.result_stack
                      if res["name"] == r["name"]), None)
        fate = engine.classify_fate(entry["params"][-1]) if entry else "n/a"
        print(f"\n_{r['name']}:")
        print(f"Peak:_{r['peak_value']:.3f}_@_t={r['peak_time']}")
        print(f"Final:_{r['final']:.3f}_Variance:_{r['variance']:.4f}")
        print(f"Oscillations:_{r['oscillations']:d}_Fate:_{fate}")

if __name__ == "__main__":
    engine = IARPhenomenologicalSimulator(dt=0.01)

    for domain, cfg in DOMAINS.items():
        for sim in cfg["sims"]:
            engine.run(sim, T=500)
            print_sim_summary(engine, domain)

    try:
        from matplotlib import pyplot as plt
        fig, axes = plt.subplots(1, 3, figsize=(18, 5))
        for ax, (domain, cfg) in zip(axes, DOMAINS.items()):
            for sim in cfg["sims"]:
                res = next((r for r in engine.result_stack
                           if r["name"] == sim.name), None)
                if res:
                    traj = res["traj"]
                    ax.plot([p[0] for p in traj], [p[1] for p in traj],
                            label=sim.name)
                    ax.set_title(cfg["title"])
                    ax.set_xlabel("Time"); ax.set_ylabel("X_C")
                    ax.legend(); ax.grid(alpha=0.3)
            plt.tight_layout(); plt.savefig("iar_simulator.png", dpi=150)
    except ImportError:
        print("(matplotlib not installed; plot skipped)")

    print("\n==_ENGINEERING_WHAT-IF_EXAMPLE:_MACHINE_HEALTH_==")
    base = Scenario("Healthy_motor", {"beta": 0.5, "theta": 0.5, "gamma": 0.7,
                                       "lam": 0.6, "eta": 0.05, "alpha": 0.5},
                   {"X_C": 0.5, "X_0": 0.5, "R": 0.0}, None)
    fatigue = Scenario("Progressive_wear", {"beta": 1.2, "theta": 0.5, "gamma": 1.0,
                                           "lam": 0.4, "eta": 0.02, "alpha": 0.5},
                      {"X_C": 0.5, "X_0": 0.5, "R": 0.0}, None)
    overload = Scenario("Overload_event", {"beta": 1.5, "theta": 0.5, "gamma": 1.2,
                                           "lam": 0.3, "eta": 0.4, "alpha": 0.5},
                       {"X_C": 0.5, "X_0": 0.5, "R": 0.0}, None)
    for s in (base, fatigue, overload):
        engine.run(s, T=500)
    print_sim_summary(engine, "Machine_Health")

```